

Database Systems Laboratory Work

Week 2: Relational Model & Keys

Name: Nurdilda Azimbekuly

Task 1

Part 1: Key Identification Exercises

Task 1.1: Superkey and Candidate Key Analysis

Given the following relations, identify all superkeys and candidate keys:

Relation A: Employee

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

Sample Data:

EmpID	SSN	Email	Phone	Name	Department	Salary
101	123-45-6789	john@company.com	555-0101	John	IT	75000
102	987-65-4321	mary@company.com	555-0102	Mary	HR	68000
103	456-78-9123	bob@company.com	555-0103	Bob	IT	72000

Your Tasks: 1. List at least 6 different superkeys 2. Identify all candidate keys 3. Which candidate key would you choose as primary key and why? 4. Can two employees have the same phone number? Justify your answer based on the data shown.

Relation B: Course Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

Business Rules:

- A student can take the same course in different semesters
- A student cannot register for the same course section in the same semester
- Each course section in a semester has a fixed credit value

Your Tasks: 1. Determine the minimum attributes needed for the primary key 2. Explain why each attribute in your primary key is necessary 3. Identify any additional candidate keys (if they exist)

1. Super keys:

1. {EmpID, Name}
2. {Email, Phone, Department}
3. {SSN, Name, Salary}
4. {Phone, Name}
5. {EmpID, Phone}
6. {Email, Name, Salary}

2. Candidate keys:

1. {EmpID}
2. {SSN}
3. {Email}
4. {Phone}

3. Which candidate key would you choose as primary key and why?:

{EmpID}, because SSN contains confidential information, while email and phone numbers may change in the future. EmpID is a stable and simple identifier for an employee.

4. Can two employees have the same phone number? Justify your answer based on the data shown.

No. Based on the data shown, no two employees have the same phone number. Each employee has a unique phone number.

Relation B: Course Registration

1. StudentID + CourseCode + Section + Semester

2. Explain why each attribute in your primary key is necessary

- **StudentID** is necessary because it identifies the student who is registering.
- **CourseCode** is necessary because it identifies the course.
- **Section** is necessary because a course can have multiple sections.
- **Semester** is necessary because a student can take the same course in different semesters.

All four attributes are necessary to uniquely identify a registration.

3. **Answer:** There are no additional candidate keys based on the given business rules. The candidate key is {StudentID, CourseCode, Section, Semester} because all four attributes are necessary to uniquely identify a registration.

1.2 . Identify all foreign key relationships

- **Enrollment.StudentID** → **Student.StudentID**
- **Enrollment.CourseID** → **Course.CourseID**
- **Student.AdvisorID** → **Professor.ProfID**
- **Department.ChairID** → **Professor.ProfID**
- **Course.DepartmentCode** → **Department.DeptCode**

Task 2.1: Hospital Management System

1)

Strong – Patients, Doctors, Departments, Appointments, Prescriptions

Weak – Hospital Rooms

2) Patients :

- PatientID – Primary key, simple
- Name – simple
- Birthdate – simple
- Address – composite: (street,city,state,zip) simple
- PhoneNumber – multi-valued simple
- Insurance – simple

Doctors:

- DoctorID – Primary
- Name – simple
- Specialization –
- PhoneNumber –
- OffLocation –

Departments:

-DepartmentCode – Primary key

-Name – simple

Simple

-Locations - simple

-Purpose – simple

Prescriptions:

-PrescriptionsID – Primary key

partial key

-Medications – simple

Foreign key

- Patients – simple

- Dosage – simple

- Instructions – simple

3) Relationships with cardinalities:

1 : 1 – we dont have this type relations

1 : N – Patient->Appointment, Doctor->Appointment, Doctor-> Prescriptions,

Patient-> Prescriptions, Department->HospitalRooms

Patient N:M Doctor (with Appointment)

4)

Appointments:

-AppID – Primary

-DoctorName –

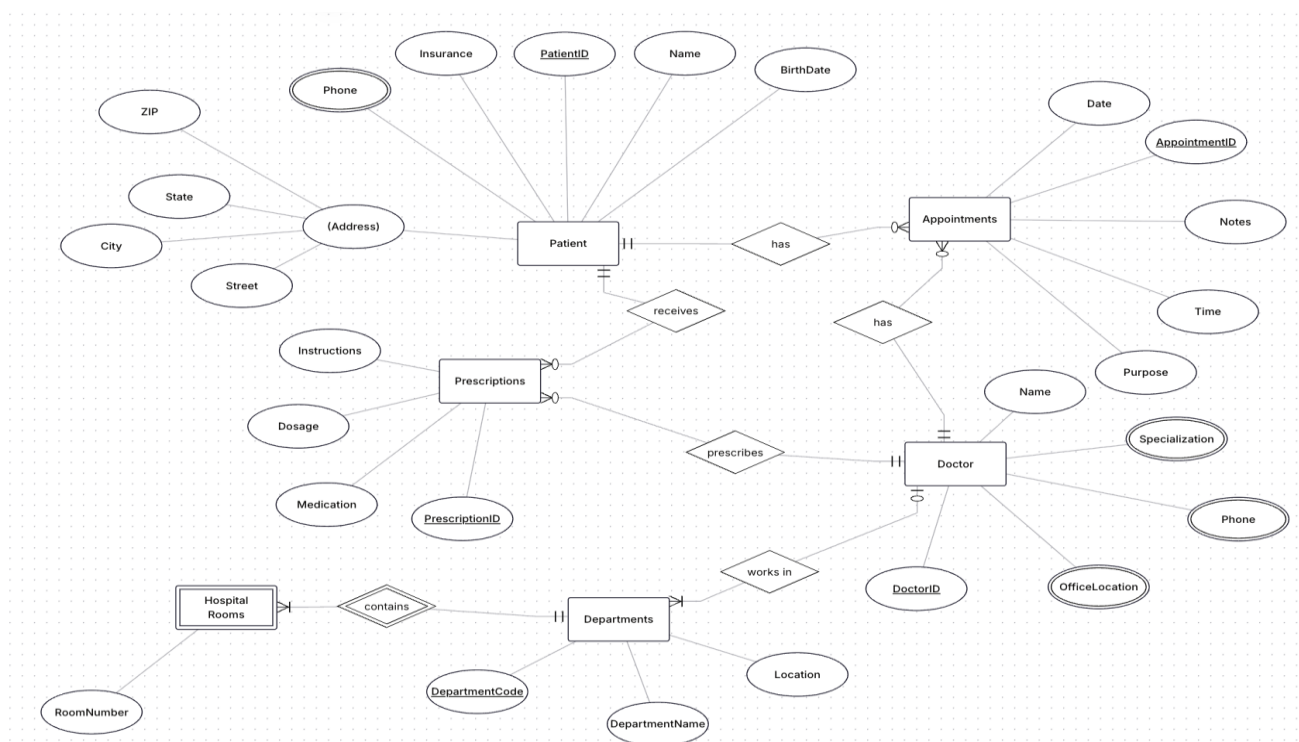
-Date - Simple

-Notes – simple

Hospital Rooms:

-RoomNumber -

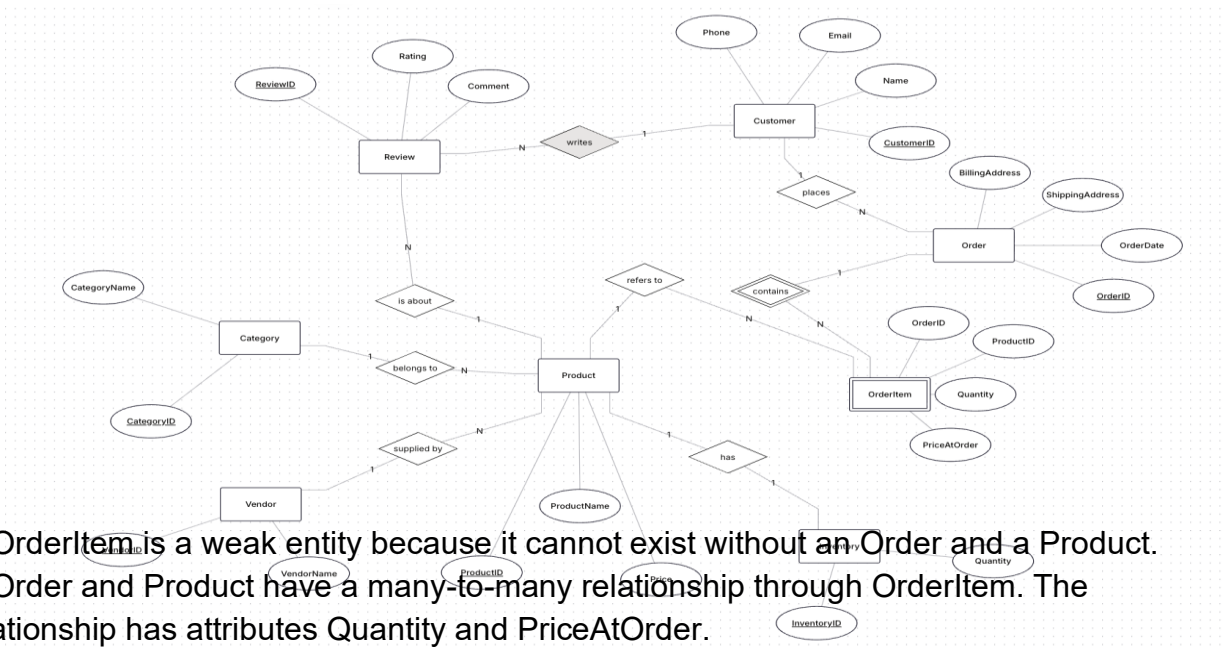
- DepartmentCode –



5) Pk are marked with underline

2.2

1)



2) OrderItem is a weak entity because it cannot exist without an Order and a Product.

3) Order and Product have a many-to-many relationship through OrderItem. The relationship has attributes Quantity and PriceAtOrder.

Task 4.1: Denormalized Table Analysis

1. Identify Functional Dependencies (FDs)

Based on standard relational semantics, the following functional dependencies exist in the table:

- **StudentID** → StudentName, StudentMajor
- **ProjectID** → ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate
- **SupervisorID** → SupervisorName, SupervisorDept
- **(StudentID, ProjectID)** → Role, HoursWorked

2. Identify Problems & Anomalies

- **Redundancy:**
 - Student profiles (Name, Major) repeat every time a student joins a new project.
 - Project and Supervisor details repeat for every student assigned to that project.

- **Insert Anomaly:** You cannot add a new student to the system until they are assigned to a project because ProjectID forms part of the primary key and cannot be null.
- **Update Anomaly:** If a supervisor changes their department, you must update every row containing a project they manage. Missing a single row causes data inconsistency.
- **Delete Anomaly:** If a student leaves their only project and the row is deleted, all records of that student's existence (Name and Major) are permanently lost.

3. Apply 1NF (First Normal Form)

- **Status:** The table is already in **1NF**, assuming that all attribute values are atomic (e.g., no comma-separated lists of multiple majors or phone numbers in a single cell).
- **Fix:** No action needed unless non-atomic data is introduced.

4. Apply 2NF (Second Normal Form)

- **Primary Key: (StudentID, ProjectID).**
- **Partial Dependencies:**
 - StudentID → StudentName, StudentMajor (Dependent only on part of the key).
 - ProjectID → ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, StartDate, EndDate (Dependent only on part of the key).
- **2NF Decomposition:** Eliminate partial dependencies by splitting them into separate relations:
 - **Student** (StudentID, StudentName, StudentMajor)
 - **Project_Temp** (ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, StartDate, EndDate)
 - **StudentProject_Assignment** (StudentID, ProjectID, Role, HoursWorked)

5. Apply 3NF (Third Normal Form)

- **Transitive Dependencies:** In Project_Temp, we have ProjectID → SupervisorID and SupervisorID → SupervisorName, SupervisorDept. Therefore, supervisor details are transitively dependent on ProjectID.
- **Final 3NF Decomposition:**
 - **Student** (StudentID, StudentName, StudentMajor)
 - **Supervisor** (SupervisorID, SupervisorName, SupervisorDept)

- **Project** (ProjectID, ProjectTitle, ProjectType, *SupervisorID*, StartDate, EndDate)
- **StudentProject_Assignment** (StudentID, ProjectID, Role, HoursWorked)

Task 4.2: Advanced Normalization

1. Determine the Primary Key

To uniquely identify a row in a student's comprehensive schedule, the primary key must be **(StudentID, CourseID, TimeSlot)**.

*Note: Because of the strict business rules, **(StudentID, Room, TimeSlot)** and **(StudentID, InstructorID, TimeSlot)** are also valid candidate keys.*

2. List All Functional Dependencies

Derived directly from the business rules:

- **StudentID**-----> StudentMajor
- **CourseID** -----> CourseName
- **InstructorID** ----->InstructorName
- **Room** -----> Building (*since rooms are unique across campus*)
- **(CourseID, TimeSlot)** -----> InstructorID, Room (*one instructor/room per section*)
- **(Room, TimeSlot)** ----->CourseID, InstructorID (*a room can only host one class at a time*)
- **(InstructorID, TimeSlot)** ----->CourseID, Room (*an instructor can only be in one place at a time*)

3. Check if the Table is in BCNF

- **Status: No**, the table is not in BCNF.
- **Reason:** For a relation to be in BCNF, the left-hand side of every non-trivial functional dependency must be a superkey. Dependencies like StudentID → StudentMajor and CourseID → CourseName violate this rule because StudentID and CourseID are not superkeys on their own.

4. Decompose to BCNF

Decomposing step-by-step to isolate independent facts results in the following **six schemas**:

1. **Student** (StudentID, StudentMajor)
2. **Course** (CourseID, CourseName)
3. **Instructor** (InstructorID, InstructorName)
4. **Facility** (Room, Building)
5. **CourseSection** (CourseID, TimeSlot, InstructorID, Room)
 - a. *Note: (Room, TimeSlot) and (InstructorID, TimeSlot) serve as alternate unique candidate keys here.*
6. **Enrollment** (StudentID, CourseID, TimeSlot)

5. Explain Potential Loss of Information / Dependencies

- **Information Loss: None.** The decomposition is completely **lossless-join**, meaning you can reconstruct the original denormalized table perfectly using natural joins without generating any spurious rows.
- **Dependency Preservation: No dependencies are lost.** Even though the relations were separated, the complex multi-attribute constraints (Room, TimeSlot $\rightarrow$... and InstructorID, TimeSlot $\rightarrow$...) all reside cleanly within the single CourseSection table, allowing them to be fully enforced via standard relational constraints.

Would you like me to proceed with another section of the lab? I can:

1. Provide the complete solution for **Part 1: Key Identification Exercises** (Employee & Course Registration relations).
2. Generate the **Relational Schema syntax (DDL)** for the normalized tables above, including all primary and foreign key constraints.
3. Help design the ER diagrams for **Part 2** (Hospital Management or E-commerce system).

Task 5.1: Real-World Application

1. Entity-Relationship (ER) Diagram Specification

To represent the system accurately based on the requirements, we define the following components using standard Crow's Foot / Chen notation concepts:

Entities and Attributes

- **Student** (Strong Entity)
 - StudentID (Primary Key - PK)
 - Name
 - Email
- **Club** (Strong Entity)
 - ClubID (PK)
 - ClubName
 - Budget
- **FacultyAdvisor** (Strong Entity)
 - AdvisorID (PK)
 - Name
 - Department
- **Event** (Strong Entity)
 - EventID (PK)
 - EventName
 - Date
 - Time
- **Room** (Strong Entity)
 - RoomNumber (PK)
 - BuildingName (PK) — *Composite PK because room numbers repeat across buildings.*
- **Expense** (Weak Entity - dependent on Club)
 - ExpenseID (Partial Key / Discriminator)
 - Amount
 - Description
 - Date

Relationships and Cardinalities

- **Club Membership: Student** (0..N) <---> (1..N) **Club** [Many-to-Many / M:N]
- **Club Leadership (Officers): Student** (0..N) <---> (0..1) **Club** [Many-to-Many / M:N] with a relationship attribute Role (e.g., President, Treasurer). *Note: A student can be an officer in multiple clubs, and a club has multiple officers.*
- **Club Advising: FacultyAdvisor** (1) <---> (0..N) **Club** [One-to-Many / 1:N]
- **Event Hosting: Club** (1) <---> (0..N) **Event** [One-to-Many / 1:N]
- **Event Attendance: Student** (0..N) <---> (0..N) **Event** [Many-to-Many / M:N]
- **Room Reservation: Event** (0..N) <---> (0..1) **Room** [Many-to-One / N:1]
- **Club Financials: Club** (1) <---> (0..N) **Expense** [One-to-Many / 1:N, Identifying relationship]

2. Normalized Relational Schema (3NF)

Below is the logical database schema mapping from the ER model. Primary keys are **underlined>** and foreign keys are *italicized*.

- **Student** (StudentID VARCHAR(10),
Name VARCHAR(100),
Email VARCHAR(100))
- **FacultyAdvisor** (AdvisorID VARCHAR(10),
Name VARCHAR(100),
Department VARCHAR(50))
- **Club** (ClubID VARCHAR(10),
ClubName VARCHAR(100),
Budget DECIMAL(10,2),
AdvisorID VARCHAR(10) NOT NULL)
 - *FK*: AdvisorID references FacultyAdvisor(AdvisorID)
- **Room** (BuildingName VARCHAR(50), RoomNumber VARCHAR(10))
- **ClubMembership** (StudentID VARCHAR(10), *ClubID* VARCHAR(10), JoinDate DATE)
 - *FK*: StudentID references Student(StudentID)
 - *FK*: ClubID references Club(ClubID)
- **ClubOfficer** (StudentID VARCHAR(10), *ClubID* VARCHAR(10), Role VARCHAR(30))
 - *FK*: StudentID references Student(StudentID)
 - *FK*: ClubID references Club(ClubID)
- **Event** (EventID VARCHAR(10), EventName VARCHAR(100), Date DATE, Time TIME, *ClubID* VARCHAR(10) NOT NULL, *BuildingName* VARCHAR(50), *RoomNumber* VARCHAR(10))
 - *FK*: ClubID references Club(ClubID)
 - *FK*: (BuildingName, RoomNumber) references Room(BuildingName, RoomNumber)
- **EventAttendance** (StudentID VARCHAR(10), *EventID* VARCHAR(10), AttendanceStatus VARCHAR(20))
 - *FK*: StudentID references Student(StudentID)
 - *FK*: EventID references Event(EventID)
- **ClubExpense** (*ClubID* VARCHAR(10), ExpenseID VARCHAR(10), Amount DECIMAL(10,2), Description TEXT, Date DATE)

- *FK*: ClubID references Club(ClubID) ON DELETE CASCADE

3. Design Decision Analysis

Scenario: Mapping the **Club Officer Positions** requirement.

- **Option A:** Add President_StudentID, Treasurer_StudentID, and Secretary_StudentID directly as foreign keys inside the Club table.
- **Option B:** Create a separate junction/bridge table ClubOfficer (StudentID, ClubID, Role).
- **My Choice & Justification:** I chose **Option B**. Option A is inflexible; it limits the system to a fixed set of three roles. If a club wants to introduce a new role (e.g., "Vice President" or "Social Media Chair"), Option A requires a schema modification (adding new columns). Option B is fully normalized and dynamic—it supports any number of custom officer roles simply by inserting a new text value into the Role attribute.

4. Three Example English Queries

- **Query 1:** "Find the total amount spent on expenses by the 'Computer Science Club' during the current semester."
- **Query 2:** "List the names and emails of all students who attended the club event titled 'Annual Hackathon' hosted on October 15th."
- **Query 3:** "Identify all faculty advisors who manage more than two student clubs simultaneously, along with the names of those clubs."